

A High-Precision and Low-Cost Approximate Transform Accelerator for Video Coding

Zhijian Hao¹, Jiaming Liu², Chenlong He², Qi Zheng², Shushi Chen², Jinchang Xu³, Yue Hao¹, Xiao Yan^{*4} and Xiaohua Ma^{*1}

¹State Key Discipline Laboratory of Wide Band-Gap Semiconductor Technology, School of Microelectronics, Xidian University

²State Key Laboratory of Integrated Chips and Systems, Fudan University, ³School of Computer Science, Peking University,

⁴Electronics Engineering Department, Xi'an University of Posts and Telecommunications

*Corresponding authors: Xiao Yan {xiao_yanxd@163.com} and Xiaohua Ma {xhma@xidian.edu.cn}

Abstract—The introduction of multiple transform types in the Versatile Video Coding (VVC) standard has yielded notable encoding gains but also imposed considerable computational burdens. Existing transform circuits of different types are typically implemented separately due to their independence, leading to substantial hardware overhead. To address this, we explore the relationship between Discrete Cosine Transform Type-2 (DCT2) and Discrete Sine Transform Type-7 (DST7) matrices and reveal a prominent diagonal aggregation phenomenon in their transfer matrix. Based on this insight, the least-squares method is applied to optimize the transfer matrix sparsity, achieving a high-precision, low-cost approximate conversion from DCT2 to DST7. Furthermore, we optimize DCT2 computation by proposing an elaborate matrix decomposition approach that allows a lightweight shift-adder unit to efficiently generate all required product terms across varying sizes. Leveraging these algorithmic optimizations, we implement a highly reusable and area-efficient approximate transform accelerator that supports sizes from 4 to 32 points and accommodates three types in VVC. Experimental results demonstrate that the proposed accelerator achieves over 44% reduction in circuit resource consumption with negligible BD-BR performance loss of just 0.53%, maintaining processing capabilities up to 8K@57 fps.

Index Terms—Video coding, transform accelerator, approximate architecture, sparsity optimization, area-efficient

I. INTRODUCTION

Transform coding has been widely integrated in video coding due to its efficient spatial decorrelation capability. Cooperating with quantization and entropy coding, it significantly improves compression rates while maintaining slight quality loss. It was proved in [1] that the Discrete Cosine Transform Type-2 (DCT2) can optimally approximate the Karhunen-Loève (K-L) transform and is also more computationally efficient, resulting in its widespread adoption in H.264 and H.265. Later, Han et al. [2] demonstrated that for images modeled by a first-order Gaussian-Markov process, the Discrete Sine Transform Type-7 (DST7) provides better decorrelation than DCT2. Therefore, the latest coding standards have introduced DST7 alongside DCT2 for more comprehensive decorrelation.

The introduction of multiple transform types provides significant coding benefits, yielding a 2.10% coding gain, but it also substantially increases computational complexity. In terms of hardware resource consumption, supporting multiple transform types requires instantiating additional transform circuits, which incurs high hardware costs. To mitigate these costs, several approaches have been proposed, grouped into three

main categories. The first category focuses on **independent optimizations for DCT2**. Early studies include the Butterfly algorithm, which reduces multipliers by leveraging symmetry. Ahmed et al. [3] further reduced multiplications by decomposing DCT matrices into sparse sub-matrices. Additionally, multiplier-less techniques [4]–[6] have been developed to optimize multiplication. Recent DCT2 hardware optimizations focus on improving reusability. Hao et al. [7] proposed using smaller transform matrices as subsets of larger ones to improve reusability. Zheng et al. [8] and Hao et al. [9] introduced data rearrangement schemes that allow even-odd decomposition circuits to be reused across different transform sizes.

The second category focuses on the **independent optimization of DST7**. Notably, Discrete Cosine Transform Type-8 (DCT8), another transform type used in VVC, exhibits significant symmetry with DST7, allowing the optimization of both transformations to be reduced to the optimization of DST7. Garrido et al. [10]–[15] explored various approaches for designing efficient DST architectures on FPGA, including utilizing DSP resources for multiplication and replacing multiplication with shift-add operations. Fan et al. [16], [17] applied the RAG-n algorithm to minimize adder count, significantly reducing resource usage.

The third category focuses on the **joint optimization of both types**. These methods typically employ approximate computation strategies to reduce resource overhead. In [18], the matrix elements are modified to multiples of 2^3 to reduce multiplier size, though suffering unneglectable performance loss. Ahmed et al. [19] proposed a method to convert results of inverse DCT2 to DST7 with good coding performance. However, it requires an inverse transform in the forward transform pipeline, making it impractical for hardware implementation.

Previous studies have predominantly focused on optimizing DCT2 or DST7 architectures in isolation, or on joint optimization strategies that are impractical for real-world applications, often failing to effectively balance performance retention with hardware constraints. In this paper, we achieve a high-precision and low-cost approximate transform accelerator designed through algorithm-hardware co-optimizations. The main contributions are summarized as follows:

- **High-precision and low-cost approximate scheme for DST7**. Our analysis reveals an efficient transfer relationship between DST7 and DCT2. Leveraging this insight,

we define a transfer matrix and further optimize its sparsity with the least squares method, facilitating a high-precision result conversion from DCT2 to DST7. Notably, 79.3% multipliers can be saved for the 32-point case.

- **Matrix decomposition-based calculation scheme for DCT2.** Building upon the butterfly algorithm, we further reduce the remaining complexity by proposing an improved computation scheme for DCT2, enabling a lightweight shift-adder unit (SAU) to generate all product terms across varying size.
- **Highly-pipelined and area-efficient approximate transform architecture.** With above algorithmic optimizations, we develop a pipelined architecture that integrates multiple resource-reuse and data-parallelism strategies. Compared to related works, the proposed architecture achieves a reduction of over 44% in resource costs and delivers the highest throughput.

II. BACKGROUND AND MOTIVATION

A. Background of transform

The transform scheme employed in VVC integrates three primary transform types: DCT2, DST7, and DCT8. By exploiting the separability, the two-dimensional (2D) transform is decomposed into two sequential 1D transforms, effectively reducing both storage and computational costs. Therefore, optimization efforts for transform architectures mainly focus on 1D transforms. The 1D DCT2 is defined as (1),

$$Y(k) = \sqrt{\frac{2}{N}} \varepsilon_k \sum_{n=0}^{N-1} x(n) \cos \left[\frac{k(2n+1)\pi}{2N} \right], \quad \varepsilon_k = \begin{cases} \sqrt{\frac{2}{N}}, & k=0 \\ 1, & k \neq 0 \end{cases} \quad (1)$$

where k is the index of transform coefficients. $Y(k)$ is the transformed outputs, $x(n)$ is the input data, and N is the transform size. It can be expressed in matrix form as follows:

$$Y = A \times X^T, \quad (2)$$

where A is the transform matrix. In practical video coding, by scaling and rounding the A matrix while preserving its orthogonality, the integer transform matrix T can be obtained. Therefore, the integer 1D transform can be conducted as (3):

$$Y = T \times X^T. \quad (3)$$

The transformation process is inherently a computation-intensive matrix multiplication. The T varies with transform types. Thus, the optimization of the transform architecture relies on refining the corresponding matrix multiplications.

B. Motivation

The integration of multiple transform types intensifies the industry's desire for a unified architecture that supports all three transform types with the resource cost of a single type. However, the independence of DCT2 and DST7 matrices poses a huge challenge as the general multipliers-based architecture leads to heavy resource overhead. While DST7 and DCT8 share the same circuit, an efficient solution for merging DCT2 and DST7 into a joint design remains unresolved. Previous

works targeting joint optimization are not efficient enough to balance coding performance with hardware friendliness, rendering them impractical for most real-world applications.

For optimizing DCT2 and DST7 jointly, we conduct an in-depth analysis of their computation processes and observe a prominent diagonal aggregation feature in the transfer matrix, motivating us to explore its sparse optimization for high-precision and cost-effective result conversion from DCT2 to DST7. Besides, we observe that after applying the butterfly transform to DCT2, the remaining matrix multiplications exhibit a beneficial feature, providing further opportunities to reduce the computational resource requirements for DCT2.

III. HARDWARE-ORIENTED ALGORITHMIC OPTIMIZATIONS

A. Algorithmic Optimizations for DST7

To establish the relationship between the DCT2 and DST7, we begin with their corresponding transform processes:

$$Y_2 = T_2 \times X^T, \quad Y_7 = T_7 \times X^T, \quad (4)$$

By transposing both sides of these equations, we obtain:

$$Y_2^T = X \times T_2^T, \quad Y_7^T = X \times T_7^T. \quad (5)$$

Further, a transfer matrix TF is defined to reflect the conversion between the results of DCT2 and DST7:

$$Y_2^T \times TF = Y_7^T \iff X \times T_2^T \times TF = X \times T_7^T, \quad (6)$$

simplifying it as:

$$T_2^T \times TF = T_7^T. \quad (7)$$

Considering that T_2 is an orthogonal matrix, we have $T_2^{-1} = \frac{T_2^T}{2^n}$, where 2^n is introduced for integer transform. Besides, the inverse matrix of T_2^T is $\frac{T_2}{2^n}$. Substitute it into (7) and TF can be derived as (9):

$$\frac{1}{2^n} \times T_2 \times T_2^T \times TF = \frac{1}{2^n} \times T_2 \times T_7^T, \quad (8)$$

$$TF = \frac{1}{2^n} \times T_2 \times T_7^T. \quad (9)$$

Fig. 1a presents the scaling and rounding case of the transfer matrix TF , which converts the transform results from DCT2 to DST7. It is evident that the elements of TF exhibit a pronounced aggregation pattern, with the larger magnitudes concentrated along the diagonal and values farther from the diagonal approaching zero. Smaller values, representing lower significance, can be discarded to reduce computational costs while preserving transfer accuracy.

The previous observation inspires the sparse optimization of TF . This optimization problem can be formulated as a constrained optimization problem, where the objective is to minimize the error defined in (10) between the original DST7 matrix T_7 and its approximation $\tilde{T}_7$ (11) computed from the sparse transfer matrix $\tilde{TF}$, subject to the constraint of limiting the number of non-zero values in each column of $\tilde{TF}$. This can be expressed mathematically as follows:

$$E(\tilde{TF}) = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} (T_7(i, j) - \tilde{T}_7(i, j))^2 \quad (10)$$

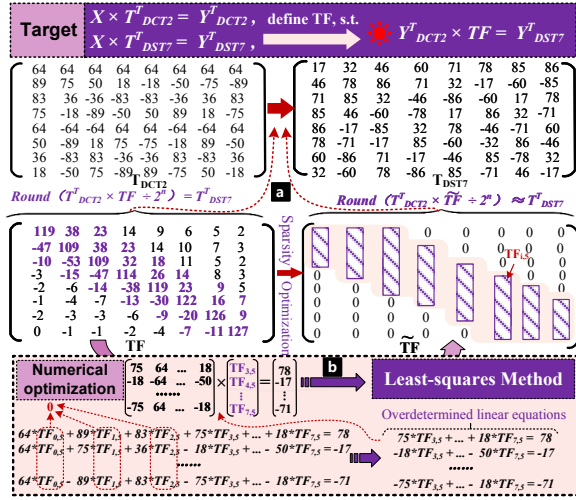


Fig. 1. The algorithmic optimizations for DST7. (a) The introduction of transfer matrix and the motivation of sparsity optimization. (b) Sparsity optimization of $TF_{i,5}$ with the least-squares method

$$\tilde{T}_7^T = \text{round} (T_2^T \times \tilde{T} \div 2^n); \quad (11)$$

$$\text{minimize } E(\tilde{T} \tilde{F}),$$

$$\text{subject to } \tilde{T} \tilde{F}_{i,j} = 0, |i - j| > (r - 1)/2, \quad (12)$$

$$\tilde{T} \tilde{F}_{i,j} \in \mathbb{Z} \cap [-2^n + 1, 2^n],$$

where r is the number of retained elements in each column of $\tilde{T} \tilde{F}$. For example, when $r = 5$, as illustrated in Fig. 1a, only the diagonal and two neighboring elements on each side are preserved. Besides, note that the error $E(\tilde{T} \tilde{F})$ is accumulated independently from every column, therefore, the optimization problem can be further decomposed into individual column-wise optimization tasks, which effectively reduces the optimization complexity. In this paper, we employ the least squares method to solve the constraint optimization for each column. Fig. 1b presents the optimization process for the sixth column $TF_{i,5}$. It is evident that, for $r = 5$, the system of equations becomes overdetermined. By formulating this system of equations as a matrix equation, it can be efficiently solved using MATLAB to obtain the least squares solution.

Following that, we conduct a group of experiments to determine the optimal variable r and the results are illustrated in Fig. 2. **For performance evaluation**, five video sequences with varying resolutions were randomly selected as test vectors, and the results of $r = 5$, $r = 7$, and $r = 9$ are depicted in Fig. 2a. As r increases, the Bjøntegaard Delta-Bit Rate (BD-BR) [20] loss decreases across all sequences. It is worth noting that the loss reduction between $r = 7$ and $r = 9$ is less significant than that between $r = 5$ and $r = 7$. Specifically, For *basketballDrill*, the performance loss decreases by 0.24% when r increases from 5 to 7, while the reduction from 7 to 9 is only 0.06%. Additionally, **the multiplier usage ratio** for the approximation to the exact transform is presented in Fig. 2b, where the multiplier usage increases approximately linearly with r . Taking performance and multiplier usage into consideration, $r = 7$ is selected as the optimal parameter.

Fig. 2c depicts the approximation of the first eight basis vectors of the 16-point DST7 when $r = 7$. The results clearly

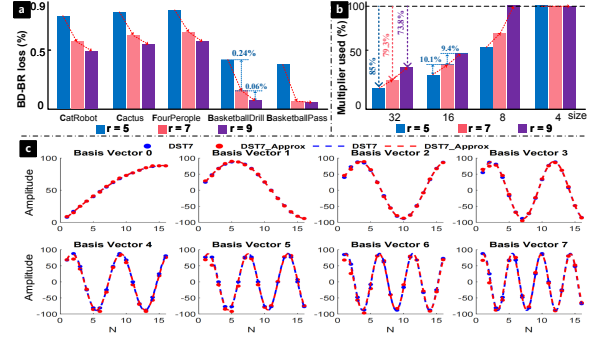


Fig. 2. (a) The Bd-BR loss of five sequences under different r . (b) The multiplier usage ratio of four sizes under different r . (c) Performance of approximation when $r = 7$.

indicate that the $\tilde{T}_7$ derived from (11) closely matches the original matrix, underscoring the efficacy of the proposed method in balancing precision and computational complexity.

B. Algorithmic Optimizations for DCT2

The algorithmic optimizations for DCT2 are inspired by an efficient hardware design of DST7. The DST7 transform matrix exhibits a beneficial feature: its row/column coefficients are derived from a unified set, differing only in position and sign. This feature enables an efficient SAU-based matrix multiplication, where N identical SAUs generate all product terms, which are then accumulated through an adder tree, significantly reducing computational resources, as shown in Fig. 3a. However, Fig. 3b reflects that this feature doesn't appear in the DCT2 transform matrix, making direct optimization impractical. Encouragingly, after applying the classic butterfly algorithm to DCT2, its sub-matrices possess this desirable feature. As shown in Fig. 3b, the 8-point DCT2 with the butterfly algorithm is presented, which merges inputs with matching coefficients to produce the E and O vectors. These vectors are subsequently processed by sub-matrices to derive the final transformed outputs. The presence of this mentioned feature in the sub-matrices allows for further optimization of the remaining calculations.

It is worth noting that directly optimizing the sub-matrix computations with SAU-based matrix multiplication still results in substantial resource consumption. This stems from the diversity in sub-matrix elements across various sizes, requiring distinct SAUs to be instantiated for each size. To enhance SAU reuse across multiple sizes, we introduce a matrix decomposition-based calculation scheme for DCT2, as depicted in Fig. 3c, defined as follows:

$$T = Q \times s + R, \quad (13)$$

here, s represents a scaling factor with a power-of-two value (2^n), determined by balancing the complexities of Q and R . As the s increases, the complexity of Q decreases and that of R increases. For each entry in $T(i, j)$, its decomposition equation is given by (14):

$$\begin{cases} T(i, j) = Q(i, j) * s + R(i, j), \\ |R(i, j)| \leq s/2. \end{cases} \quad (14)$$

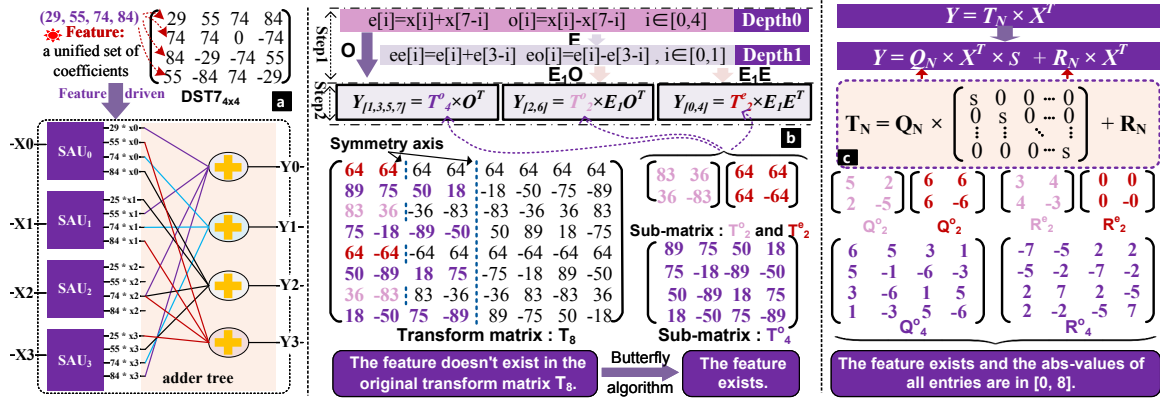


Fig. 3. (a) The beneficial feature and the SAU-based matrix multiplication. (b) The butterfly algorithm and sub-matrix of DCT2. (c) The proposed calculation scheme based on matrix decomposition.

Given that the maximum entry of the DST7 and DCT2 matrix is 90. We can acquire the range of absolute value in Q and R matrices presented in Table I. Clearly, when s is equal to 16, these two matrices share a similar level of complexity. In this case, the optimized calculation scheme for DCT2 can be described as (15):

$$Y^T = Q \times X^T \times s + R \times X^T, \quad (15)$$

where the X is the inputs, and the computationally intensive matrix operation is carried out by two simpler matrix multiplications and shift operations. The limited range of Q and R enables a lightweight SAU circuit with output $\{1x, 2x, 3x, 4x, 5x, 6x, 7x, 8x\}$ to generate all required product terms, improving the reusability dramatically.

IV. THE HARDWARE TRANSFORM ACCELERATOR IMPLEMENTATION

A. Overall Architecture Design

We implement a highly parallel, extensively reusable, and deeply pipelined transform accelerator in Fig. 4, capable of processing 32 samples per cycle. For transform sizes below 32 points, multiple input sets are combined to create a 32-point input. This approach processes 8 groups of 4-point inputs, 4 groups of 8-point inputs, 2 groups of 16-point inputs, or a single group of 32-point inputs in each cycle. Such parallel processing achieves the highest level of concurrency among existing research.

The proposed accelerator features a deeply pipelined architecture with six stages: even-odd decomposition, lightweight SAU calculation, selector tree, adder tree, DCT2 result integration, and transfer calculation. The initial five stages focus on DCT2, whereas the last stage handles DST7. The following sections provide a detailed description of the data flow within each pipeline stage, highlighting the strategies for parallelism and reusability.

TABLE I
THE RANGE OF ABSOLUTE VALUE IN Q AND R MATRICES

Scale factor s	4	8	16	32	64
abs Q range	[0,23]	[0,11]	[0,6]	[0,3]	[0,1]
abs R range	[0,2]	[0,4]	[0,8]	[0,16]	[0,32]

B. Even-Odd Decomposition Stage

Stage0 is the even-odd decomposition stage, breaking down inputs into E and O vectors via the butterfly algorithm, as shown in Fig. 3b. For this purpose, a custom decomposition circuit was designed, specifically for processing 8 groups of 4-point inputs, as illustrated in Fig. 4b.

Reusability. Applying this decomposition circuit to other input sizes directly leads to data mismatches. For instance, with a set of 32-point input, the correct $e[0]$ should be derived by adding X_1 and X_{31} , but the circuit would erroneously compute it as the sum of X_1 and X_3 , leading to a malfunction. To ensure functional correctness and enable reuse of the decomposition circuit across varying input sizes, we propose a data rearrangement scheme depicted in Fig. 5, where X_{31} is repositioned to the location of X_3 accordingly. This strategy eliminates data mismatches and allows a single decomposition circuit to support four input sizes, effectively reducing the adder resource consumption by 75%.

C. Lightweight SAU Calculation Stage

Stage1 is the SAU computation stage, generating all the product terms involved in the matrix multiplications.

Reusability. As shown in Table I, the absolute value range of all matrix elements is confined within $[0, 8]$. Therefore, we design a lightweight SAU embedding with only three adders to efficiently generate the product terms, as shown in Fig. 4a. The reuse strategy of this pipeline stage is twofold: first, the SAU can be reused across different transform sizes; second, there is no need to instantiate separate SAU circuits for the Q and R sub-matrices, as they share a similar value range.

Parallelism. The accelerator instantiates 32 lightweight SAUs in total, which operate in parallel to generate the product terms corresponding to the 32 inputs.

D. Selector Tree

Stage2 is the selector tree stage, which routes the product terms produced by the SAUs. The appropriate SAU outputs are selected based on the matrix elements and forwarded to the next stage via Adder-tree Selectors (ATS), as depicted in Fig. 4c. Each ATS includes multiple selectors, controlled by the *size* signal.

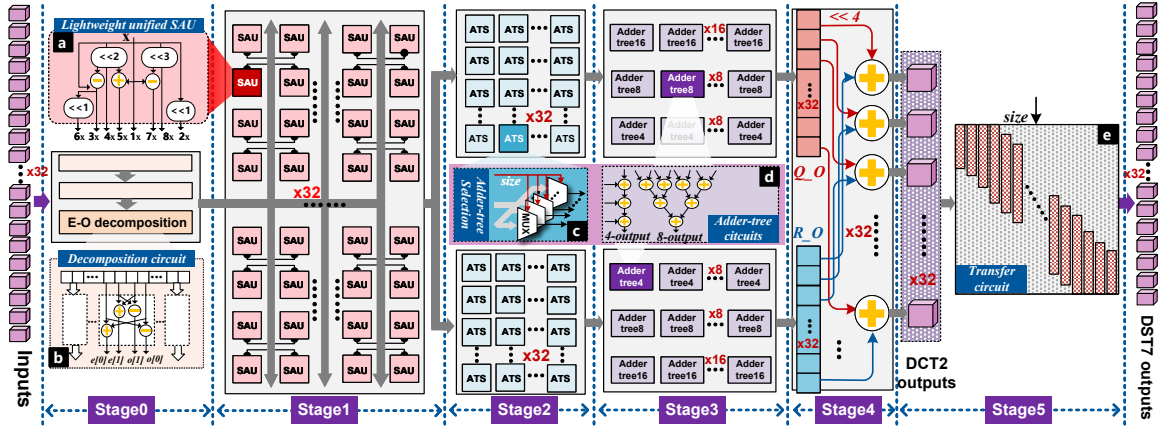


Fig. 4. The proposed architecture. (a) Lightweight unified SAU. (b) Decomposition circuit. (c) Adder tree selection. (d) Adder-tree circuit. (e) Transfer circuit.

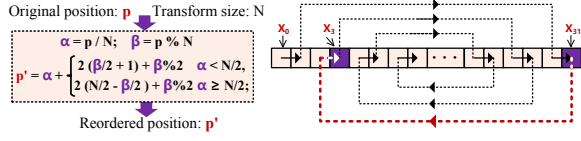


Fig. 5. The proposed data rearrangement scheme.

Parallelism. In this stage, the selection process for the sub-matrices Q and R is performed in parallel. Each of them requires 32 ATS units, resulting in a total of 64 ATS units working concurrently.

E. Adder Tree and DCT2 Result Integration Stages

Stage3 is the adder tree stage, where the selected product terms are accumulated.

Parallelism. This stage employs three types of adder trees: adder-tree4, adder-tree8, and adder-tree16, as depicted in Fig. 4d for efficiently accumulating 4, 8, and 16 inputs, respectively. In the original DCT2, each output is obtained by accumulating 32 product terms, which would typically demanding 32 adder-tree32. However, the butterfly algorithm reduces the number of multiplications significantly. For the 32-point case, after the even-odd decomposition, the actual sub-matrices involved in the computation are O , EO , EEO , and EEE , with dimensions of 16×16 , 8×8 , 4×4 and 4×4 , respectively. Consequently, instead of 32 adder-tree32, only 16 adder-tree16, 8 adder-tree8, and 8 adder-tree4 are needed. Meanwhile, the sub-matrices Q and R are processed concurrently, the number of adder-tree circuits is doubled. Thus, the proposed accelerator requires 32 adder-tree16, 16 adder-tree8, and 16 adder-tree4 for parallel accumulation.

Stage4 is the DCT2 result integration stage, where the final DCT2 results are computed using the outputs of sub-matrices Q and R , as defined in (15).

F. Transfer Calculation Stage

Stage5 is the transfer calculation stage, where the DCT2 results are approximated to DST7 results through sparse matrix multiplication, as defined in (6). This stage employs 212 multipliers, significantly fewer than $32 \times 32 = 1024$ multipliers needed for precise DST7 calculation. This approximation strategy achieves a 79.3% reduction in multiplier consumption.

TABLE II
COMPARISON OF THE ALGORITHMS

	DCT2	TCSVT [19]	ICIP [21]	ISCAS [18]	Ours
Performance	Evaluating metric: BD-BR (%)				
Hardware Friendly	✓	✗	✓	✓	✓
A1 & A2	1.36	0.20	1.13	1.79	0.69
B	1.54	0.14	1.32	1.16	0.74
C	2.97	0.06	0.96	0.93	0.23
D	2.37	-	0.99	0.96	0.35
E	2.24	0.18	0.95	1.25	0.65
Average	2.10	0.15	1.07	1.22	0.53

V. EVALUATION

A. Experimental Setup

Performance evaluations were carried out using the VVC reference software VTM17.0 under all-intra configurations, employing BD-BR as the evaluation metric. The accelerator was designed in Verilog HDL and synthesized using Global Foundry's 28nm standard cell library to measure area, frequency, and power consumption metrics.

B. Coding performance comparison

To validate the effectiveness of the proposed algorithm, a comparison with other approximate transformation algorithms was conducted, as shown in Table II. When the transformation type in VVC is restricted to DCT2, coding performance drops by 2.10% relative to the use of multiple transform types. Significantly, applying the proposed approximation method within the multiple transform framework minimizes this loss, reducing it to just 0.53%. This ensures that most of the coding benefits of the original approach are maintained.

Approximation algorithms in existing research can be divided into two types. **Neglecting hardware constraints** and **Hardware-friendly algorithms**. The first type often overlooks practical hardware limitations in algorithm design. For example, [19] proposes a high-precision private approximation architecture. Despite its high accuracy, it necessitates embedding an inverse transform circuit in the forward pipeline and modifying the decoder, which doubles the circuit area and renders it unsuitable for most embedded applications. The second type, including [18], [21], and the proposed algorithm, is tailored to be hardware-friendly, allowing easy

TABLE III
COMPARISON OF DIFFERENT TRANSFORM ARCHITECTURES

Design	TCSVT [22]	TCSVT [23]	Ours	TCSVT [16]	ISCAS [18]	Ours	TCE [24]	TVLSI [8]	ICASSP [25]	Ours
Technology(nm)	90	90	28	65	65	28	90	65	28	28
Frequency (MHz)	250	187	769	384	333	769	345	476	600	769
Parallelism	8	16	32	32	2~8	32	8	32	2	32
Gate Count	163K	88K	132K	275.4K	30.7K	51.9K	136.4K	329K	96.8K	184.4K
Power (mW)	33.72	17.04	42	68.6	NA	16.5	NA	95.2	NA	58.7
NGC	20.375	5.5	4.125	8.61	3.838 ~ 15.35	1.628	7.05	10.281	48.4	5.763
NP	0.135	0.091	0.054	0.179	-	0.021	-	0.2	-	0.076
Sizes	4~32	4~32	4~32	4~32	4~32	4~32	4,8	4~32	4~64	4~32
Types	DCT2	DCT2	DCT2	DCT8 DST7	DCT8 DST7	DCT8 DST7	DCT2/5/8 DST1/7	DCT2 DST7/DCT8	DCT2 DST7/DCT8	DCT2 DST7/DCT8

integration into the forward transform pipeline without decoder modifications. While [18] and [21] are hardware-efficient, they result in coding performance losses of over 1%. In comparison, our method cuts this loss in half, highlighting its enhanced approximation accuracy.

C. Hardware Implementation Comparison

To demonstrate the hardware superiority of the proposed accelerator, a comparison with existing works is presented in Table III, highlighting resource requirements, throughput, and power metrics.

Resource. With a parallelism of 32 samples per cycle and a gate count of 184.4K, the proposed accelerator significantly outperforms existing designs. Compared to [8], which requires 329K gates for the same parallelism, our design reduces resource usage by 44%. Moreover, while [16] supports only two transform types at the same throughput with 275.4K gates, our accelerator achieves a 33% reduction in gate count despite supporting more transform types, thanks to advanced algorithmic optimizations.

Since different designs have varying specifications, the level of data parallelism they support may differ. To facilitate a fairer comparison, we introduce the Normalized equivalent Gate Count (NGC), defined in (16), as a metric for evaluating design efficiency. Then, we provide a comparison of the NGC for DCT2 and DST7/DCT8 across various works.

$$NGC = Gate\ Count / Parallelism . \quad (16)$$

For DCT2, our accelerator achieves a 32-point data parallelism with only 132.5K equivalent gates, resulting in the NGC of 4.125. The NGC of works [22] and [23] in the comparison list are 20.375 and 5.5, respectively. Compared to them, our accelerator reduces the normalized resources for DCT2 by **77.9%** and **25%**, respectively.

For DST7/DCT8, the computational resources in our work are primarily allocated to the transfer calculation stage, requiring 51.9K gates with an NGC of 1.628. Two designs, [16] and [18], are compared. The design in [16] achieves an NGC of 8.61 with 275.4K gates for 32-point parallelism. Compared to this, our design reduces normalized resources by 81.1%. For work [18], which supports variable throughput with a maximum parallelism of 8 points and an NGC ranging from

3.838 to 15.35, our accelerator achieves a normalized resource reduction of **57.6%** to **89.4%**.

The designs in [24], [8], and [25] support three transform types in VVC, with NGC values of 17.05, 10.281, and 48.4, respectively. The proposed design, which also supports three transform types with a constant and maximum parallelism of 32-point, achieves an NGC of 5.763. This represents a reduction in normalized resources of **66.2%**, **44%**, and **88%** compared to these works.

Throughput. Beyond area efficiency, the proposed architecture also delivers the highest processing speed among all designs. It achieves superior parallelism and operating frequency compared to existing works, making it capable of supporting real-time encoding for **8K@57fps**.

Power. The power efficiency of the proposed accelerator is evaluated using frequency-normalized dynamic power (NP) as defined in [22]. Table III indicates that, compared to the works reporting power consumption, the proposed accelerator achieves NP savings of over **40.7%**, **88.3%**, and **62%** for DCT2, DST7, and three types, respectively.

VI. CONCLUSION

This paper presents a high-precision, cost-effective approximate transform accelerator, supporting three transform types and sizes ranging from 4 to 32 points. Following the principle of algorithm-hardware co-optimization, an approximate algorithm based on a sparse transfer matrix is proposed for DST7, and a matrix decomposition-based algorithm is proposed for DCT2. Furthermore, based on these two algorithms, a deeply pipelined, area-efficient, and highly parallel accelerator is designed. Evaluation results demonstrate that the proposed design offers significant advantages in resource, throughput, and power consumption.

ACKNOWLEDGMENT

This work is supported by the China Postdoctoral Science Foundation under Grant Number 2024M762536 and National Natural Science Foundation of China under Grant 62304179. Corresponding authors: Xiao Yan and Xiaohua Ma.

REFERENCES

- [1] R. Clarke, "Relation between the karhunen loève and cosine transforms," *IEE Proceedings F (Communications, Radar and Signal Processing)*, vol. 128, pp. 359–360(1), November 1981.

- [2] J. Han, A. Saxena, V. Melkote, and K. Rose, "Jointly optimized spatial prediction and block transform for video and image coding," *IEEE Transactions on Image Processing*, vol. 21, no. 4, pp. 1874–1884, 2012.
- [3] A. Ahmed, M. U. Shahid, and A. u. Rehman, "N point dct vlsi architecture for emerging hevc standard," *VLSI Design*, vol. 2012, no. 1, p. 752024, 2012.
- [4] W. Zhao, T. Onoye, and T. Song, "High-performance multiplierless transform architecture for hevc," in *2013 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2013, pp. 1668–1671.
- [5] M. Masera, G. Masera, and M. Martina, "An area-efficient variable-size fixed-point dct architecture for hevc encoding," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 1, pp. 232–242, 2020.
- [6] P. K. Meher, S. Y. Park, B. K. Mohanty, K. S. Lim, and C. Yeo, "Efficient integer dct architectures for hevc," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 24, no. 1, pp. 168–178, 2014.
- [7] Z. Hao, F. Xu, G. Xiang, P. Zhang, X. Zeng, and Y. Fan, "A multiplierless transform architecture with the diagonal data mapping transpose memory for the avs3 standard," in *2021 IEEE 14th International Conference on ASIC (ASICON)*, 2021, pp. 1–4.
- [8] Z. Hao, H. Sun, G. Xiang, P. Zhang, X. Zeng, and Y. Fan, "A reconfigurable multiple transform selection architecture for vvc," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 5, pp. 658–669, 2023.
- [9] M. Zheng, J. Zheng, Z. Chen, L. Wu, X. Yang, and N. Ling, "A reconfigurable architecture for discrete cosine transform in video coding," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 3, pp. 810–821, 2020.
- [10] M. J. Garrido, F. Pescador, M. Chavarrías, P. J. Lobo, and C. Sanz, "A high performance fpga-based architecture for the future video coding adaptive multiple core transform," *IEEE Transactions on Consumer Electronics*, vol. 64, no. 1, pp. 53–60, 2018.
- [11] M. J. Garrido, F. Pescador, M. Chavarrías, P. J. Lobo, C. Sanz, and P. Paz, "An fpga-based architecture for the versatile video coding multiple transform selection core," *IEEE Access*, vol. 8, pp. 81 887–81 903, 2020.
- [12] M. J. Garrido, F. Pescador, M. Chavarrías, P. J. Lobo, and C. Sanz, "A 2-d multiple transform processor for the versatile video coding standard," *IEEE Transactions on Consumer Electronics*, vol. 65, no. 3, pp. 274–283, 2019.
- [13] A. Kammoun, S. Ben Jdidia, F. Belghith, W. Hamidouche, J. F. Nezan, and N. Masmoudi, "An optimized hardware implementation of 4-point adaptive multiple transform design for post-hevc," in *2018 4th International Conference on Advanced Technologies for Signal and Image Processing (ATSIP)*, 2018, pp. 1–6.
- [14] A. Kammoun, W. Hamidouche, F. Belghith, J.-F. Nezan, and N. Masmoudi, "Hardware design and implementation of adaptive multiple transforms for the versatile video coding standard," *IEEE Transactions on Consumer Electronics*, vol. 64, no. 4, pp. 424–432, 2018.
- [15] Z. Zhang, X. Zhao, X. Li, L. Li, Y. Luo, S. Liu, and Z. Li, "Fast dst-vii/dct-viii with dual implementation support for versatile video coding," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 1, pp. 355–371, 2021.
- [16] Y. Fan, Y. Zeng, H. Sun, J. Katto, and X. Zeng, "A pipelined 2d transform architecture supporting mixed block sizes for the vvc standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 9, pp. 3289–3295, 2020.
- [17] F. Yibo, K. Jiro, S. Heming, Z. Xiaoyang, and Z. Yixuan, "A minimal adder-oriented 1d dst-vii/dct-viii hardware implementation for vvc standard," in *2019 32nd IEEE International System-on-Chip Conference (SOCC)*, 2019, pp. 176–180.
- [18] Y. Zeng, H. Sun, J. Katto, and Y. Fan, "Approximated reconfigurable transform architecture for vvc," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021, pp. 1–5.
- [19] A. Kammoun, W. Hamidouche, P. Philippe, O. Déforges, F. Belghith, N. Masmoudi, and J.-F. Nezan, "Forward-inverse 2d hardware implementation of approximate transform core for the vvc standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 11, pp. 4340–4354, 2020.
- [20] G. Bjøntegaard, "Calculation of average psnr differences between rd-curves," 2001. [Online]. Available: <https://api.semanticscholar.org/CorpusID:61598325>
- [21] L. Chang, Z. Liu, X. Ji, and D. Wang, "Coding sensitive based approximation algorithm for power efficient vbs-dct vlsi design in hevc hardwired intra encoder," in *2017 IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3001–3005.
- [22] M. Masera, M. Martina, and G. Masera, "Adaptive approximated dct architectures for hevc," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 27, no. 12, pp. 2714–2725, 2017.
- [23] M. Masera, G. Masera, and M. Martina, "An area-efficient variable-size fixed-point dct architecture for hevc encoding," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 1, pp. 232–242, 2020.
- [24] A. C. Mert, E. Kalali, and I. Hamzaoglu, "High performance 2d transform hardware for future video coding," *IEEE Transactions on Consumer Electronics*, vol. 63, no. 2, pp. 117–125, 2017.
- [25] I. Farhat, W. Hamidouche, A. Grill, D. Menard, and O. Déforges, "Lightweight hardware implementation of vvc transform block for asic decoder," in *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 1663–1667.